Better agents through feedback

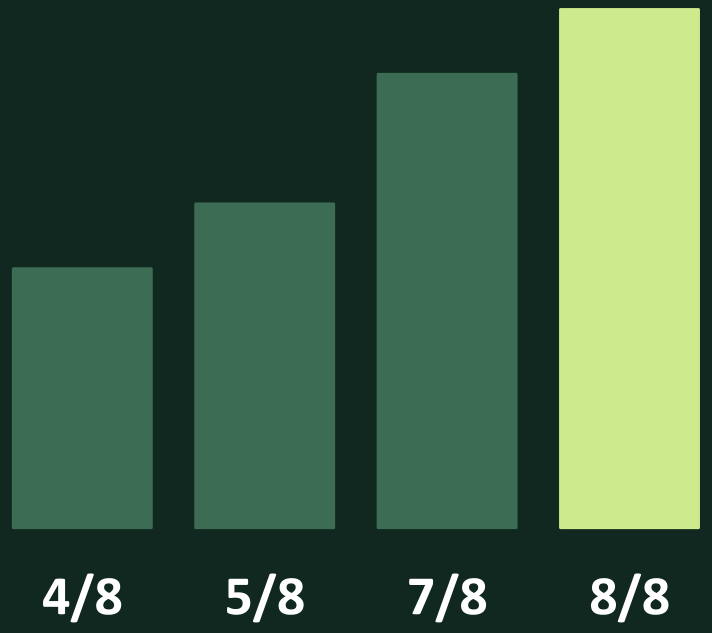
A stepping stone toward recursive self-improvement

BOUNDED STRATEGY OPTIMIZATION

Not open-ended RSI. Not model training. Not ASI.

Observed development results

50% to 100%



One deterministic program. Three predefined settings.

A stepping stone, not open-ended RSI

The distinction: a better task policy versus a better process for producing future improvements.

IMPLEMENTED

Improve the task policy

The same reflection algorithm selects better values for three routing settings.

Policy changes. The improver stays fixed.

NOT IMPLEMENTED

Improve the improver itself

Stronger RSI would improve mechanisms that drive later improvements, then independently validate that benefit.

No such recursive capability growth here.

No autonomous knowledge invention / No model-weight training / No ASI claim

Sources: agent.py; improve.py; docs/EVALUATION.md

The evaluator supplies the answer key

Route each ticket to billing, account, security, or other. Humans authored the expected labels.

DEVELOPMENT

8 Errors guide proposals; accuracy must improve.

REGRESSION GUARD

4 Previously correct cases must remain correct.

NON-SELECTION AUDIT

6 Separate reporting only; never selects a candidate.

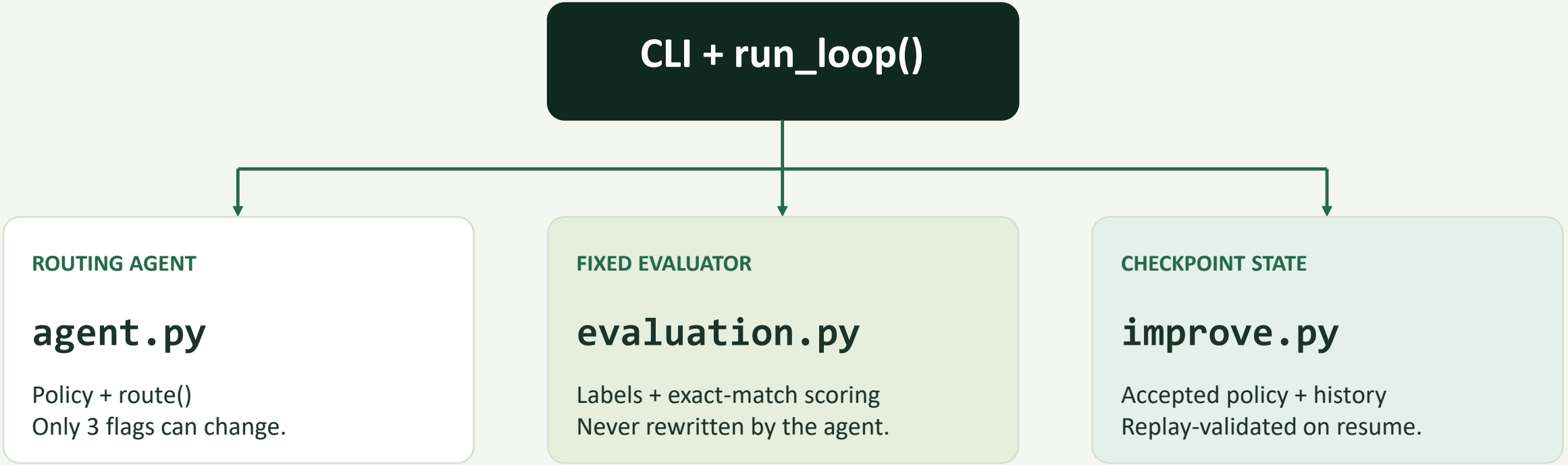
HUMAN-AUTHORED EXPECTATION

I need an INVOICE	**billing**
My account was hacked; please refund the invoice	**security**
This charger is broken	**other**

Source: evaluation.py | Exact-match accuracy; equal weight per case.

One controller, three clear responsibilities

These are modules and functions inside one deterministic program, not autonomous LLM agents.



main() reports the separate audit after the loop. It does not use audit scores to promote policies.

Sources: agent.py; evaluation.py; improve.py

The six-stage feedback loop

A proposed setting is not adopted until it passes the complete fixed selection suite.



Repeat from the accepted policy until: perfect development score, total proposal limit, or no useful candidate.

Source: improve.py | Each proposal consumes the cumulative budget, whether accepted or rejected.

What reflect() actually discovers

Count repaired failures for each disabled flag; choose the largest count, then perform full evaluation.

Current policy	Normalize	Whole word	Security first	Chosen
Baseline	1	1	1	Normalize
+ normalization	on	1	2	Security
+ security priority	on	1	on	Whole word

Numbers are failures repaired, not a complete candidate score. "on" means already enabled.

FIXED TIE-BREAK

1 Normalize case
2 Whole words
3 Security priority

No LLM reflection.
No new vocabulary.
No self-written optimizer.

This is greedy local search. Neutral or harmful intermediate changes are not explored.

Source: improve.py reflect(); probe counts recomputed from the replay-validated run.

Three policy changes repair three error types

Each new policy retains earlier accepted settings. The expected labels stay unchanged.

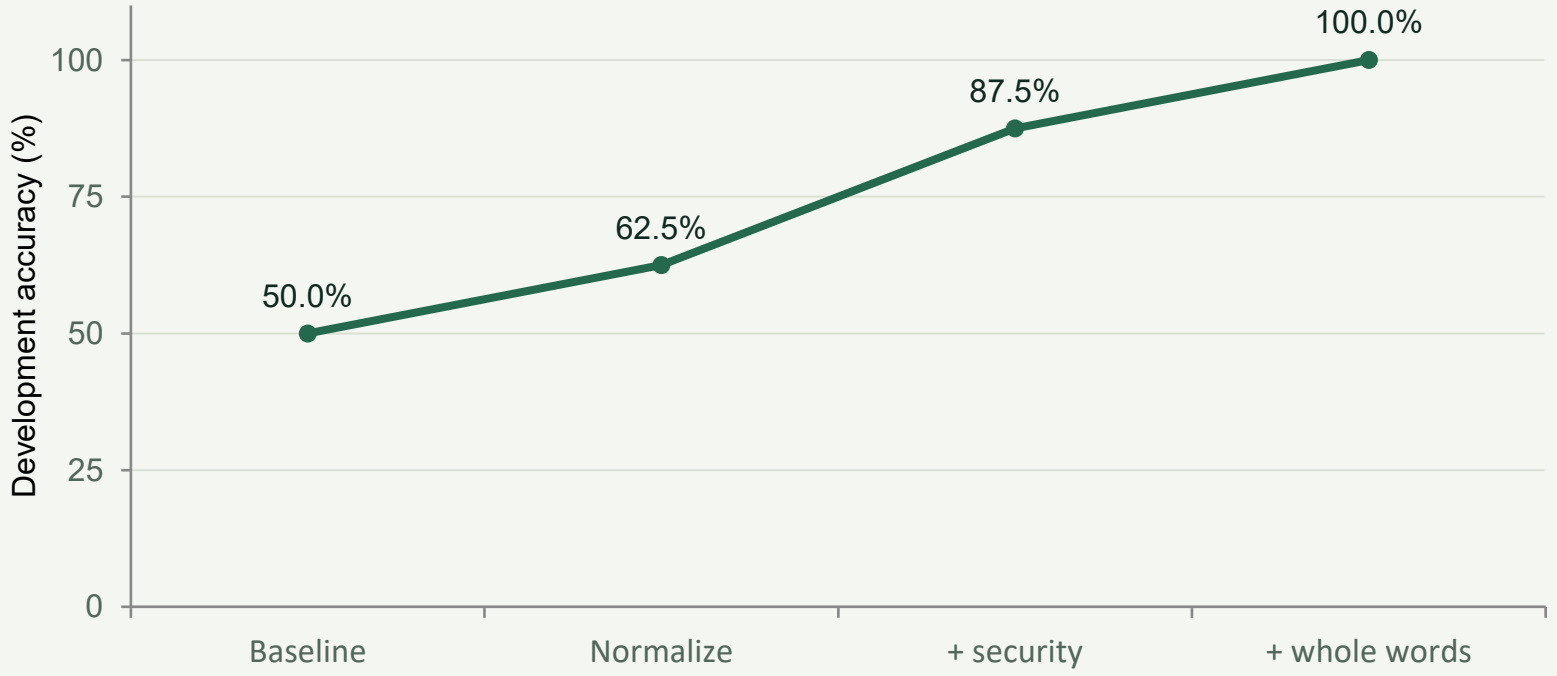
01	Normalize case	**I need an INVOICE** Case folding exposes the invoice keyword.	other → billing
02	Security first	**My account was hacked; please refund the invoice** Security takes priority over a billing keyword.	billing → security
03	Whole words	**This charger is broken** Do not match charge inside charger.	billing → other

Security-first repairs a second case too: HACKED account; INVOICE appeared.

Source: evaluation.py development cases; before/after predictions reproduced from saved policies.

Progress is measured, not scripted

Observed execution on November 2, 2026. Three proposals accepted; zero rejected in this run.



REGRESSION GUARD

4/4 throughout

NON-SELECTION AUDIT

2/6 to 6/6

Tiny and public;
not proof of generalization.

Development counts: 4/8 → 5/8 → 7/8 → 8/8. Stop reason: target_reached.

Source: saved execution checkpoint | Commit 742e63c | Native chart data is editable.

Only validated progress carries forward

An aggregate score gain cannot hide a previously correct case becoming wrong.

PROMOTION GATE

Strictly higher development accuracy

AND

No previously correct case regresses
on development or regression suites

PERSISTED STATE

```
accepted policy
candidate predictions + rationale
accept/reject decisions + reasons
history + suite fingerprint
current termination reason
```

If rejected: keep the previous policy, record the rejection, and consume budget. No rejection occurred in this run.

Source: improve.py acceptance(), make_record(), save_state(), load_state()

Resume a policy, not a background learner

A checkpoint carries successful behavior forward. The candidate budget is cumulative across resumed runs.

PARTIAL RUN

```
python improve.py --state .improvement\partial.json `
  --max-iterations 1
```

RESUME WITH A LARGER TOTAL CEILING

```
python improve.py --state .improvement\partial.json `
  --max-iterations 6
```

`target_reached`

No development failures

`iteration_limit`

Total proposal budget reached

`no_candidate`

No useful untried single-flag change

New path = fresh run. Existing path = verified resume. Invalid history fails explicitly, rather than silently resetting.

Sources: README.md; improve.py run_loop() and CLI

Toward stronger RSI: improve the improver

Research question: can a changed improvement mechanism produce better later improvements on fresh tasks?

01

Task-policy adaptation

Implemented here

Choose better predefined routing settings.

02

Improve search

Not implemented

Propose and compare changes to reflection or search itself.

03

Test the new improver

Not implemented

Test whether the changed improver helps on fresh tasks and budgets.

Required controls: isolated execution / independent evaluation / fresh tasks / permission review / rollback

This slide describes proposed future work, not capabilities implemented in this repository.

Keep the improvement loop inspectable

The demonstrated capability is narrow, measurable policy adaptation, not open-ended recursive self-improvement.

MEASURE	Fixed expectations expose actual errors.
PROMOTE	Only non-regressing gains are accepted.
REUSE	Verified state carries the policy forward.

SOURCE MAP

Task behavior
agent.py

Fixed labels + splits
evaluation.py

Reflect / gate / state
improve.py

Limits + run instructions
docs/EVALUATION.md

```
python improve.py --state .improvement\fresh-demo.json --max-iterations 6
```